

CMPU3010 Databases 2

Getting Started with Docker

You can use Docker Desktop to run the PostgreSQL and pgAdmin environment required for this module on your own computer.

You do **not** need to become a Docker expert. For this module, you initially only need to be able to:

- start Docker Desktop;
- open a terminal in the correct folder;
- run a supplied docker-compose.yml file;
- check that the containers are running;
- stop and restart the containers.

Docker Desktop includes Docker Compose, so you do not need to install Docker Compose separately if you download this.

If Docker is new to you, these short official resources explain the main concepts:

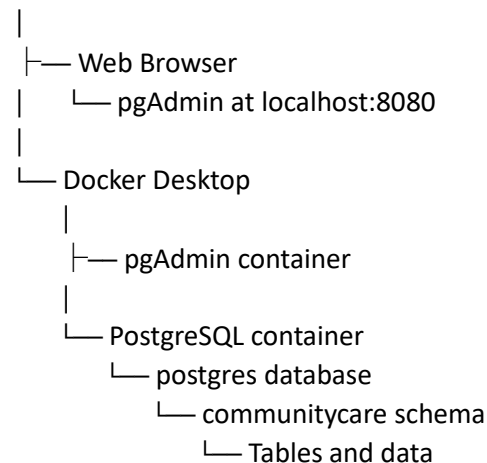
[Docker — What is a container?](#)

[Docker — What is Docker Compose?](#)

For CMPU3010, Docker is a tool we are using to provide the database environment. Learning Docker itself is not the objective of the module.

What we are aiming to achieve is the following:

Your Computer



1. Install Docker Desktop

If Docker Desktop is not already installed, download and install it for your operating system.

Windows

- Windows users can find detailed installation instructions here:
[Docker Desktop for Windows](#)

- On Windows, Docker Desktop normally uses **WSL 2** to run Linux containers. You may need to enable WSL 2 and hardware virtualisation as part of the installation. If you need to do this then the docker installation instructions cover it.
- After installation, start Docker Desktop using **Run as administrator**, particularly if you receive a system privileges error.

macOS

- Docker Desktop is available for both **Apple silicon (M-series)** and **Intel-based Macs**. Make sure you download the version appropriate for your Mac:
[Docker Desktop for Mac](#)
- After installation, start **Docker Desktop** from the Applications folder and wait for Docker to finish starting before continuing.

Linux

- Linux users can install **Docker Desktop for Linux**. Installation instructions differ depending on your Linux distribution:
[Docker Desktop for Linux](#)
- Experienced Linux users may already have **Docker Engine** and **Docker Compose** installed and can use these directly without Docker Desktop.

If you cannot use Docker Desktop

- If you cannot install or run Docker Desktop on **Windows, macOS or Linux**, you can use [Supabase](#) instead.
- Supabase provides an **online PostgreSQL environment**, so you do not need to run PostgreSQL or Docker on your own computer. You can connect to your Supabase PostgreSQL database using **DBeaver** and complete the PostgreSQL work for this module in much the same way as students using a local Docker environment.
- Instructions for setting up and connecting to Supabase are provided separately.

2. Start Docker/Docker Desktop

Before running the lab files, make sure Docker is running on your computer.

Windows users

- Find **Docker Desktop** in the Windows Start menu.
- Right-click **Docker Desktop** and select **Run as administrator**.
- This is particularly important if Docker Desktop reports that it cannot start because of **system privileges**.
- Wait until Docker Desktop has fully started before continuing.

macOS users

- Open **Docker Desktop** from the **Applications** folder, Launchpad, or Spotlight.
- Wait until Docker Desktop has fully started before continuing. You should see the Docker icon in the menu bar when Docker Desktop is running.
- You do not normally need to run Docker Desktop with administrator privileges. macOS may ask you to enter your password or approve permissions during installation or first startup.

Linux users

- If you installed **Docker Desktop for Linux**, start Docker Desktop from your applications menu and wait until it has fully started.
- If you are using **Docker Engine** rather than Docker Desktop, make sure the Docker service is running. You can check using:
`docker --version`
and:
`docker ps`
- If you receive a permissions error when running Docker commands, follow the Docker Linux post-installation instructions rather than routinely running all Docker commands using `sudo`.
[Docker Linux post-installation instructions](#)

3. Download the Lab Files

Download the files supplied for the lab.

Your folder should look similar to:

```
Lab01/
├──
├── docker-compose.yml
├── 01-communitycare-schema.sql
└── 02-communitycare-data.sql
```

The supplied `docker-compose.yml` file tells Docker which containers we want to create and how they should be configured.

You **do not need to modify this file** unless instructed to do so.

4. Start the Containers

- Open a Terminal/Command Prompt in the Lab Folder
 - Your terminal needs to be in the folder containing `docker-compose.yml`.
- Make sure **Docker Desktop is running**.
 - From the folder containing `docker-compose.yml`, run:
`docker compose up -d`
 - This means:
 - **docker compose** — use Docker Compose;
 - **up** — create and start the services defined in `docker-compose.yml`;
 - **-d** — run them in the background.
 - More information is available here: [Docker documentation: docker compose up](#)
 - The first time you run this command, Docker may need to download the required images. This may take a few minutes.

5. Check That the Containers Are Running

Run: `docker compose ps`

You should see the containers created by your `docker-compose.yml` file and their current status.

You can also open: **Docker Desktop → Containers** and check the containers there.

If the required containers are shown as running, your Docker environment is ready.

6. Connect using PGAdmin

Docker is now running your PostgreSQL environment.

The supplied Docker environment includes pgAdmin. Open your web browser and go to:

`http://localhost:8080`

Log in to pgAdmin using the details supplied in `docker-compose.yml`.

Open pgAdmin.

Create a server:

- **Host:** postgres
- **Port:** 5432
- **Database:** postgres
- **Username:** postgres
- **Password:** postgres

Open the query tool and run the sql in `01-communitycare-schema.sql`.

Then run the sql in `02-communitycare-data.sql`.

Check that the data has loaded, for example `select * from appointment;`

Useful Docker Commands

You only need a small number of Docker commands for this module.

Command	What it does
<code>docker compose up -d</code>	Creates and starts the containers
<code>docker compose ps</code>	Shows the status of the containers
<code>docker compose logs</code>	Displays messages/logs from the containers
<code>docker compose down</code>	Stops and removes the containers

Normally, if you are simply finished working for your containers, you can use:

`docker compose down`

Be Careful with -v. The `docker compose down -v` option removes the Docker volumes associated with the environment. This can delete your PostgreSQL database data. Only use this command if you have specifically been instructed to reset the database environment.

When you want to use the environment again:

1. Start **Docker Desktop**.
2. Open a terminal in the folder containing `docker-compose.yml`.
3. Run: `docker compose up -d`

Troubleshooting

Docker Desktop will not start

- **Windows**
 - Close Docker Desktop.
 - Find **Docker Desktop** in the Start menu.
 - Right-click it and select **Run as administrator**.
 - Wait for Docker Desktop to finish starting.
 - If Docker reports a problem with **WSL 2** or **virtualisation**, consult:
[Docker Desktop for Windows — installation and requirements](#)
- **macOS**
 - Quit Docker Desktop completely.
 - Open **Docker Desktop** again from the Applications folder.
 - If macOS requests permission or your administrator password, follow the on-screen instructions.
 - Wait for Docker Desktop to finish starting.
 - If Docker Desktop still does not start, check that your version of macOS and your Mac hardware are supported:
[Docker Desktop for Mac — installation and requirements](#)
- **Linux**
 - If you are using Docker Desktop, close and restart it from your applications menu.
 - If you are using **Docker Engine** rather than Docker Desktop, check that the Docker service is running. For example: `sudo systemctl status docker`
 - If necessary, start it using: `sudo systemctl start docker`
 - If Docker commands produce permission errors, consult:
[Docker Linux post-installation instructions](#)
- **docker is not recognised or the command is not found**
 - If you see an error such as:
'docker' is not recognized... or: docker: command not found
 - check that:
 - Docker has been installed correctly;
 - Docker Desktop is running, if you are using Docker Desktop;
 - you have closed and reopened your terminal since installing Docker.
 - Then try: `docker --version`
 - If the problem continues, restart your computer and try again.
- **Docker cannot find docker-compose.yml**
 - Make sure your terminal is in the folder containing the supplied docker-compose.yml file.
On **Windows**, use: `dir`
On **macOS or Linux**, use: `ls`
 - You should be able to see docker-compose.yml
 - If you cannot see the file, use `cd` to move to the correct folder.
 - Once you can see docker-compose.yml, run: `docker compose up -d`
 - **Note:** The command is `docker compose`, with a space. You may find older tutorials that use `docker-compose` with a hyphen.

- **docker compose up -d gives a permission error**
 - **Windows:** Make sure Docker Desktop is running. If necessary, close it and restart it using **Run as administrator**.
 - **macOS:** Check whether Docker Desktop is asking you to approve permissions or enter your administrator password.
 - **Linux:** If you are using Docker Engine and receive an error such as *permission denied while trying to connect to the Docker daemon*, follow Docker's Linux post-installation instructions rather than routinely running all of your Docker commands with sudo.
 - [Docker Linux post-installation instructions](#)
- **A container does not start or stops unexpectedly**
 - Check the status of the containers: `docker compose ps`
 - Then examine the messages produced by the containers: `docker compose logs`
 - The logs will often indicate what has gone wrong.
 - For example, you may see that a required port is already being used by another application.
- **A port is already in use**
 - If Docker reports that it cannot bind to a port or that the **port is already allocated**, another application may already be using the port specified in `docker-compose.yml`.
 - For this module, a common example would be **port 5432**, which is normally used by PostgreSQL. This can happen if you already have PostgreSQL installed and running directly on your computer.
 - Do not change the supplied `docker-compose.yml` file unless instructed to do so. If this occurs in the lab, ask your lecturer or teaching assistant for assistance.